WEEK 3 — AWS NETWORKING & DATABASES

VPC • Public & Private Subnets • EC2 • Security Groups • Amazon RDS

Student: Aditya Shukla **Program:** SkillNexis AWS Internship **Region:** US East (N. Virginia), us-east-1

Objective. Build a basic AWS network with public and private subnets, launch EC2 instances, configure traffic rules, create a managed MySQL database with Amazon RDS, and verify that a PHP application hosted on EC2 can connect to RDS.

1. VPC and Network Foundation

A dedicated VPC named **week3-vpc** was created. The workflow completed DNS hostname and DNS resolution settings, two subnets, an internet gateway, route tables, a route and route-table associations. This establishes a foundation for separating public-facing resources from private resources.

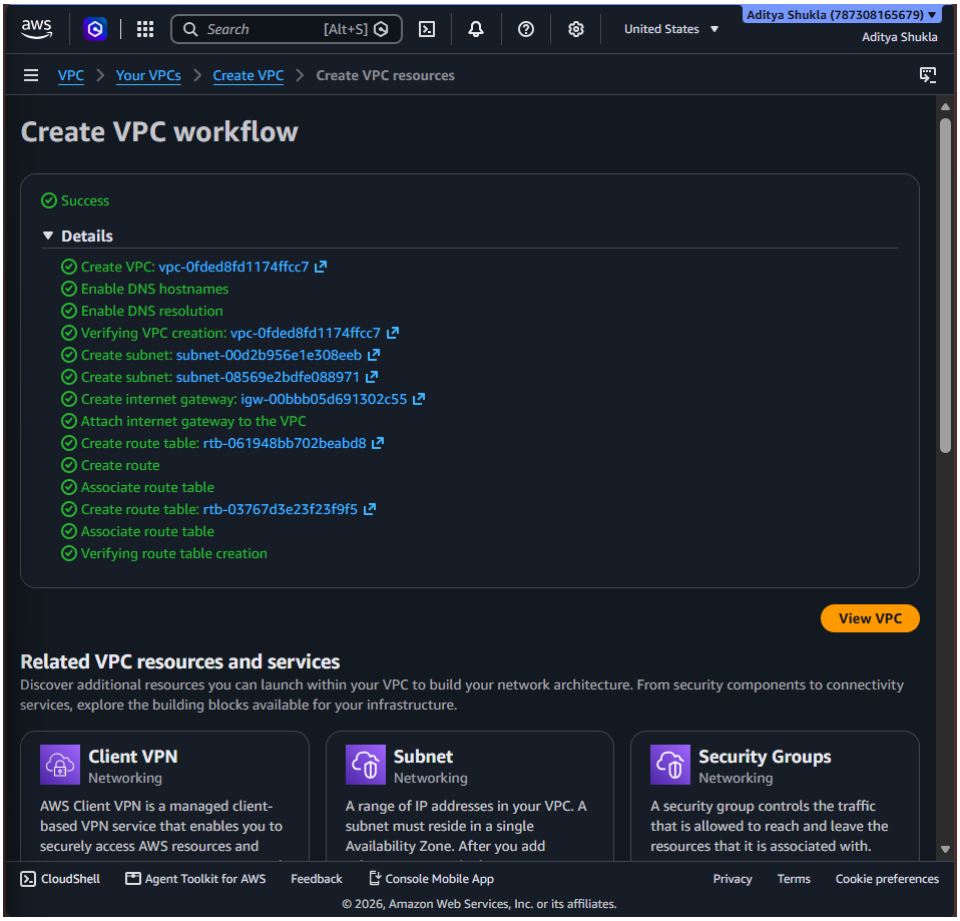


Figure 1 — AWS VPC creation workflow completed successfully, including subnet and route-table resources.

Network design intent: the public subnet is intended for resources that need public reachability; the private subnet is intended for resources that should not receive direct inbound internet traffic. Security groups provide instance-level traffic filtering.

2. EC2 Instances in Public and Private Subnets

Two EC2 instances were launched in the new VPC to demonstrate subnet placement. The console evidence shows one instance in the public subnet and another in the private subnet.

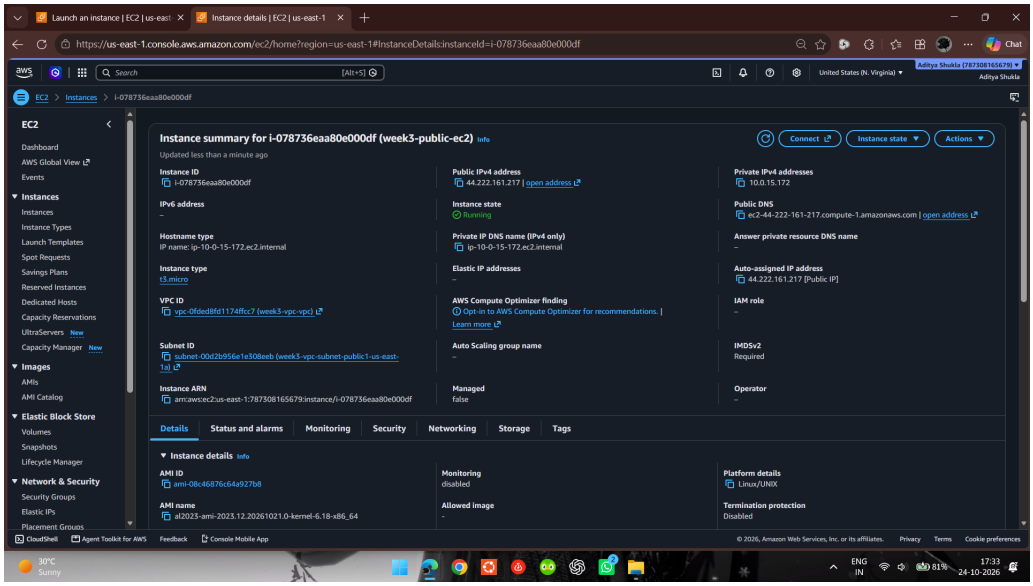


Figure 2 — EC2 instance launched in the public subnet (week3-public-ec2).

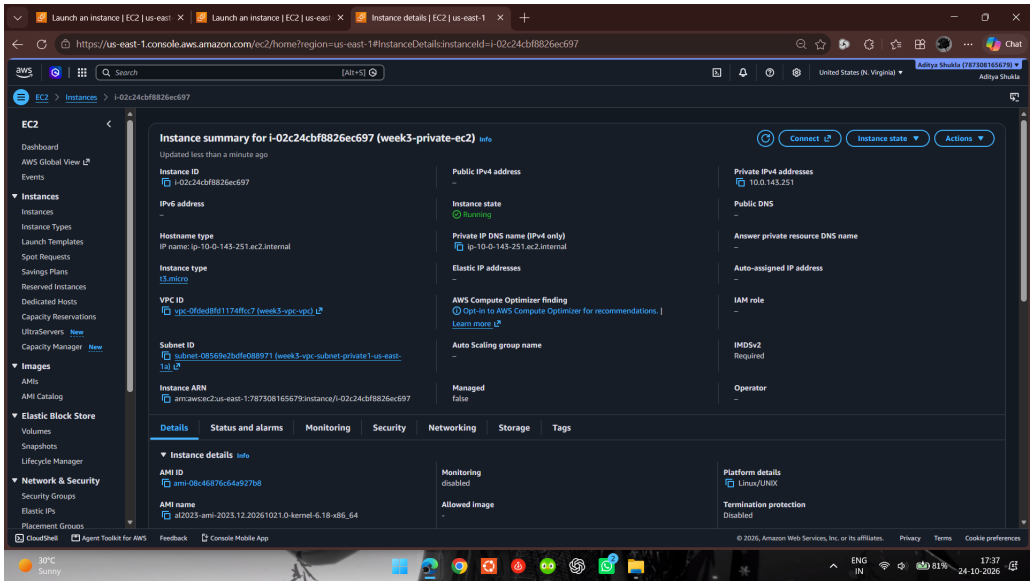


Figure 3 — EC2 instance launched in the private subnet (week3-private-ec2).

Resource	Observed configuration / purpose
Public EC2	Placed in the public subnet; the instance summary shows a public IPv4 address.
Private EC2	Placed in the private subnet; the instance summary shows no public IPv4 address.
VPC	Both instance summaries reference the week3-vpc VPC.

3. Configure Inbound and Outbound Traffic Rules

Security group rules control which network traffic is allowed to reach an instance and which traffic can leave it. The evidence below records the inbound and outbound rules configured for the private-subnet security group.

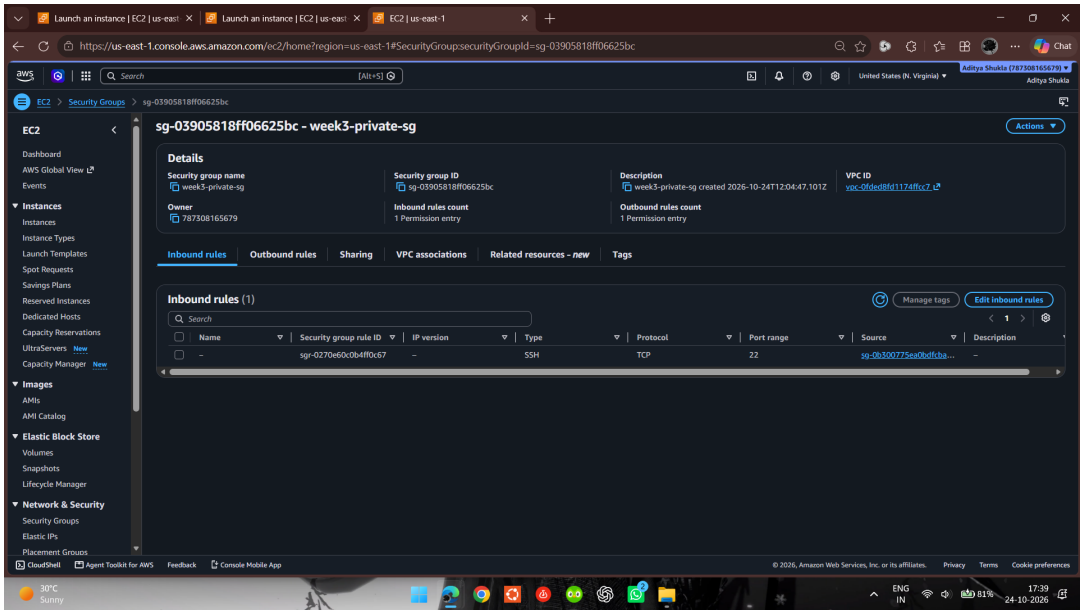


Figure 4 — Inbound rule shown for SSH over TCP port 22, sourced from a security-group reference.

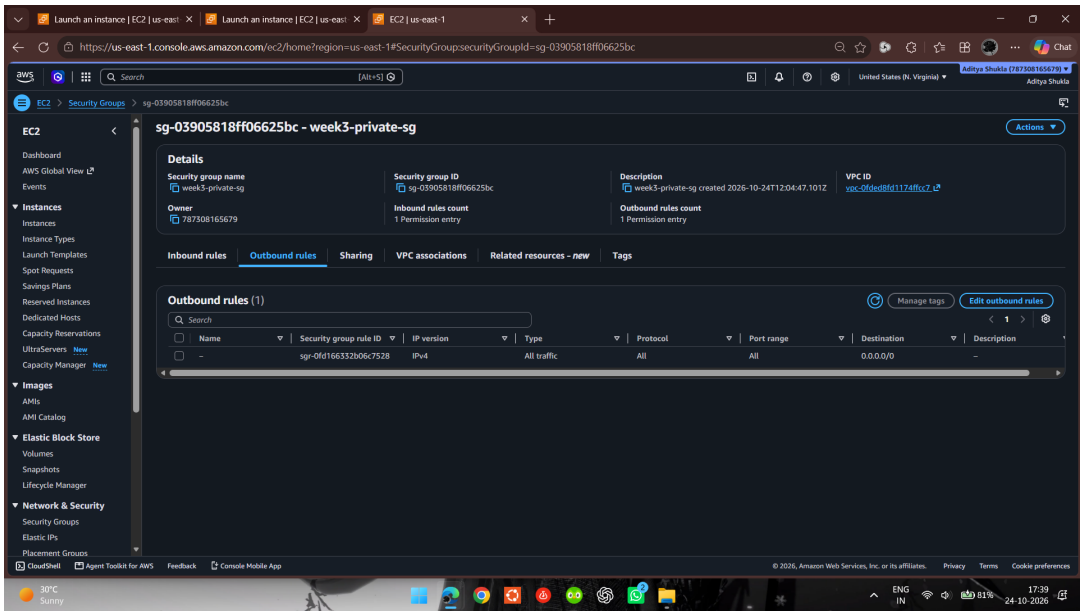


Figure 5 — Outbound rule allows IPv4 traffic to 0.0.0.0/0.

Rule summary

Direction	Configured rule	Purpose
Inbound	SSH / TCP / port 22; source is a security-group reference	Limits SSH to the referenced source instead of all IP addresses.
Outbound	All IPv4 traffic to 0.0.0.0/0	Allows outbound connections, subject to other network controls.

Security note: limit inbound rules to the smallest necessary source range. For temporary SSH testing, use **My IP** rather than leaving port 22 open to the whole internet.

4. Create an Amazon RDS MySQL Database

A managed MySQL database named **skillnexis-db** was created in Amazon RDS. The console confirms that the database instance is available and uses the MySQL Community engine. It was configured as a small single-AZ learning database.

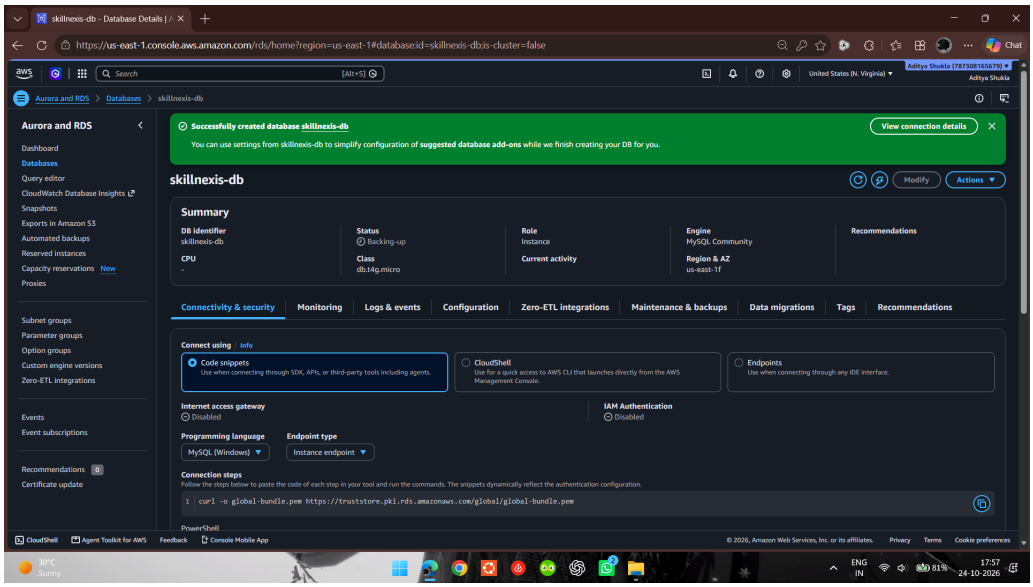


Figure 6 — RDS database skillnexis-db created successfully and shown as Available.

5. Connect the RDS Database to EC2

The AWS connection setup workflow successfully configured the connection between RDS database **skillnexis-db** and EC2 instance **skillnexis-ec2**. The setup associates security groups so the application host can reach MySQL on port 3306 without exposing the database directly to the public internet.

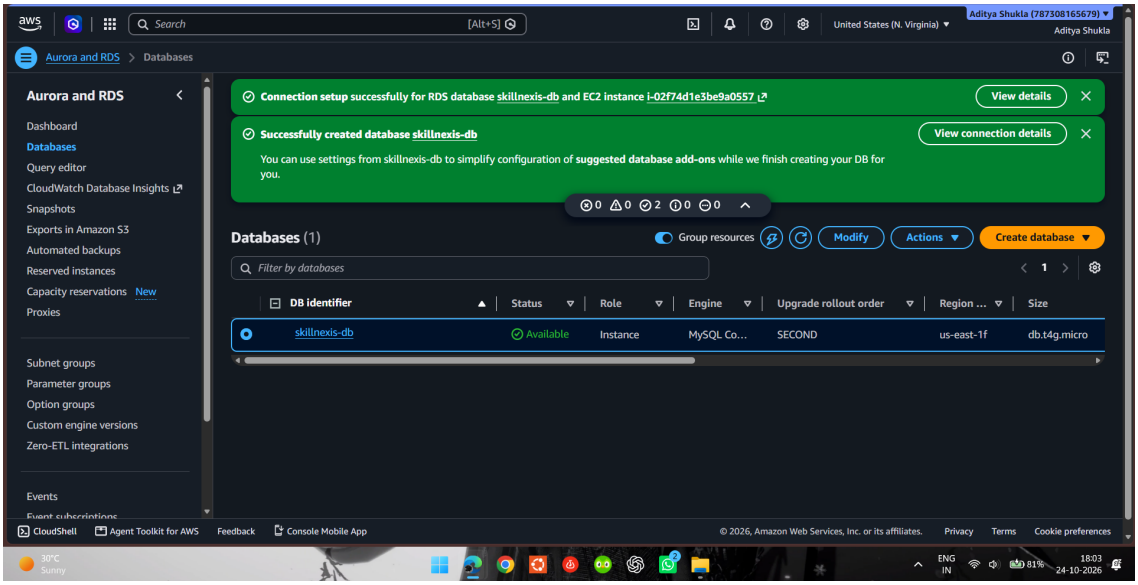


Figure 7 — AWS confirmation that RDS-to-EC2 connection setup completed successfully.

Implementation note: the subnet-placement exercise and the RDS application-connection demonstration were completed as separate lab setups. This report does not imply that the RDS-connected application host was one of the two week3-vpc subnet instances.

6. Verify the EC2-to-RDS Connection

The EC2 terminal connected to the RDS MySQL endpoint using the database administrator account. The MariaDB monitor welcome message and a successful connection ID confirmed that the EC2 host could reach the database and authenticate.

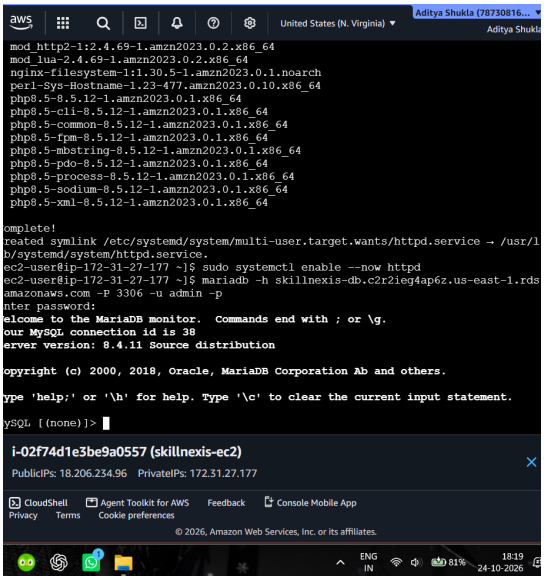


Figure 8 — Terminal login to the RDS MySQL endpoint succeeded.

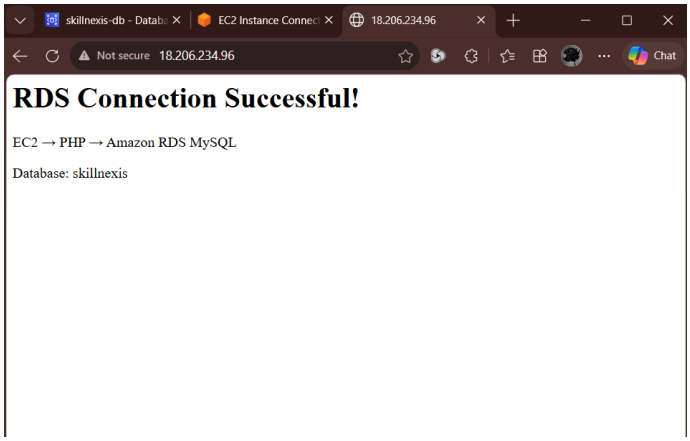


Figure 9 — PHP sample application displays "RDS Connection Successful!".

End-to-end application flow

Stage	Evidence / result
Web server	Apache was installed and served the PHP page over HTTP from the EC2 public IP.
PHP application	The sample application used PHP's MySQLi extension to open a database connection.
Amazon RDS	The application connected to the skillnexis database hosted on the RDS MySQL instance.
Verification	The browser displayed "RDS Connection Successful!" after the connection check passed.

Assignment completion summary

Requirement	Status
VPC with public and private subnets	Completed — VPC, subnets, internet gateway and route-table associations created.
EC2 instances in subnets	Completed — public-subnet and private-subnet instances shown in the console.
Inbound and outbound rules	Completed — security-group rule evidence captured.
MySQL RDS connected to EC2-hosted app	Completed — terminal login and PHP browser verification succeeded.

Conclusion. Week 3 networking and database tasks were completed and documented with AWS console, terminal and browser evidence. **Cost reminder:** stop or delete test resources when no longer needed; stopping EC2 does not necessarily stop RDS or storage charges.